Mentimeter

Laboratorul 7

Overriding, overloading & Visitor pattern

Mentimeter

Instructions

Mentimeter

Overloading

- mai multe metode cu acelasi nume dar cu semnaturi diferite
- semnatura = nume metoda + numarul si tipul parametrilor
- + exceptii

Mentimeter

Tipul de return nu face parte din semnatura unei functii

Mentimeter

```
abstract class Student {
    public abstract void solve(String problem);
    public abstract int solve(String problem);
}
```

Error duplicate method X

Mentimeter

Overloading ok

→ modific fie tipul parametrilor, fie numarul lor

Mentimeter

Supraincarcare = polimorfism static

- static = se face la compilare
- compilatorul alege metoda pe baza tipului referinterelor (la runtime, stim deja ce metoda apelam)

Mentimeter

Overriding

- aceleasi metode (aceeasi semnatura + acelasi tip de return) implementate diferit la copii

Mentimeter

Overriding = polimorfism dinamic

- alegem metoda la runtime
- numai la runtime o sa stim exact ce tip are o instanta

Mentimeter

Polimorfismul dinamic

- abilitatea unei clase să se comporte ca o altă clasă de pe lanțul de moștenire

Mentimeter

Reguli suprascriere 1

- aceeasi semnatura
- acelasi tip de return sau un tip care mosteneste tipul initial
- specificatorul de acces nu poate fi mai restrictiv

Mentimeter

Reguli suprascriere 2

- metodele de static si final nu pot fi suprascrise
- constructorii nu pot fi suprascrisi
- nu poate arunca mai multe exceptii sau exceptii mai generale

Mentimeter

```java
class Human {
    public void solve(String problem) {
        System.out.println("of");
    }
}

class Student extends Human {
    public void solve(String problem) {
        System.out.println("of student");
    }
}
```

Exemplu overriding

Mentimeter

Adnotarea @Override

- complet optionala
- daar ajuta: cod mai usor de citit + compilatorul verifica tipul de return si parametrii functiei

Mentimeter

int != Integer

- void f(int x) vs void f(Integer x)
- Integer permite si null, int nu
- (int = tip primitiv, Integer = wrapper)

Mentimeter

super

- constructorul din clasa parinte: super(....)
- metoda X din clasa parinte: super.x(...)

Mentimeter

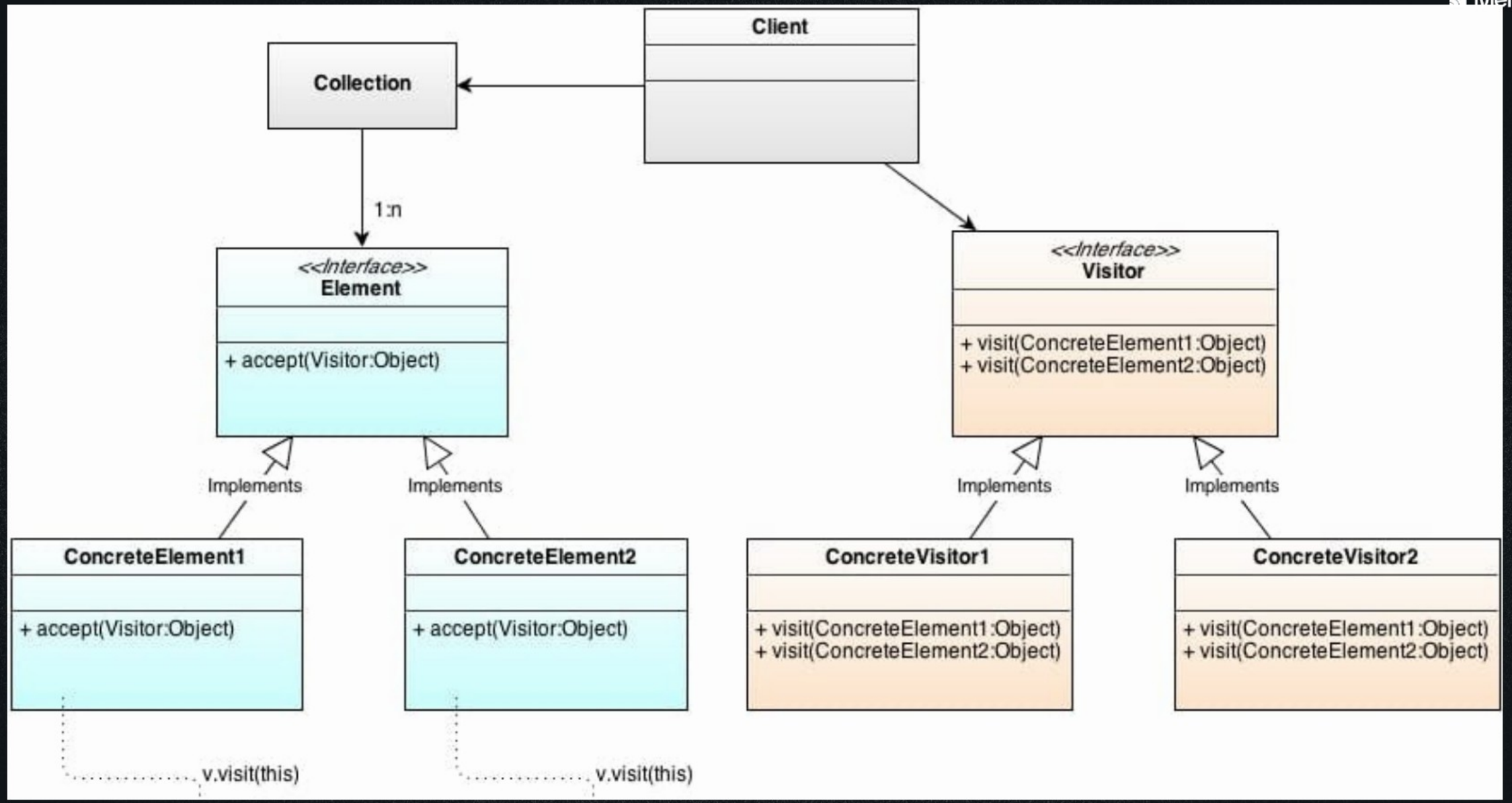
Design patterns

- modeleaza relatiile si interactiunile dintre clase

Mentimeter

Visitor

Mentimeter



Structura

Mentimeter

Visitor

- separa algoritmul de structura pe care opereaza
- putem adauga noi functionalitati de prelucrare fara a modifica structura
- pattern behavioral - comunicare intre obiecte

Mentimeter

Cand folosim Visitor?

- operatii diferite pe tipuri diferite de obiecte
- schimbarea/adaugarea operatiilor fara a modifica clasele

Mentimeter

Cand nu e recomandat sa folosim Visitor?

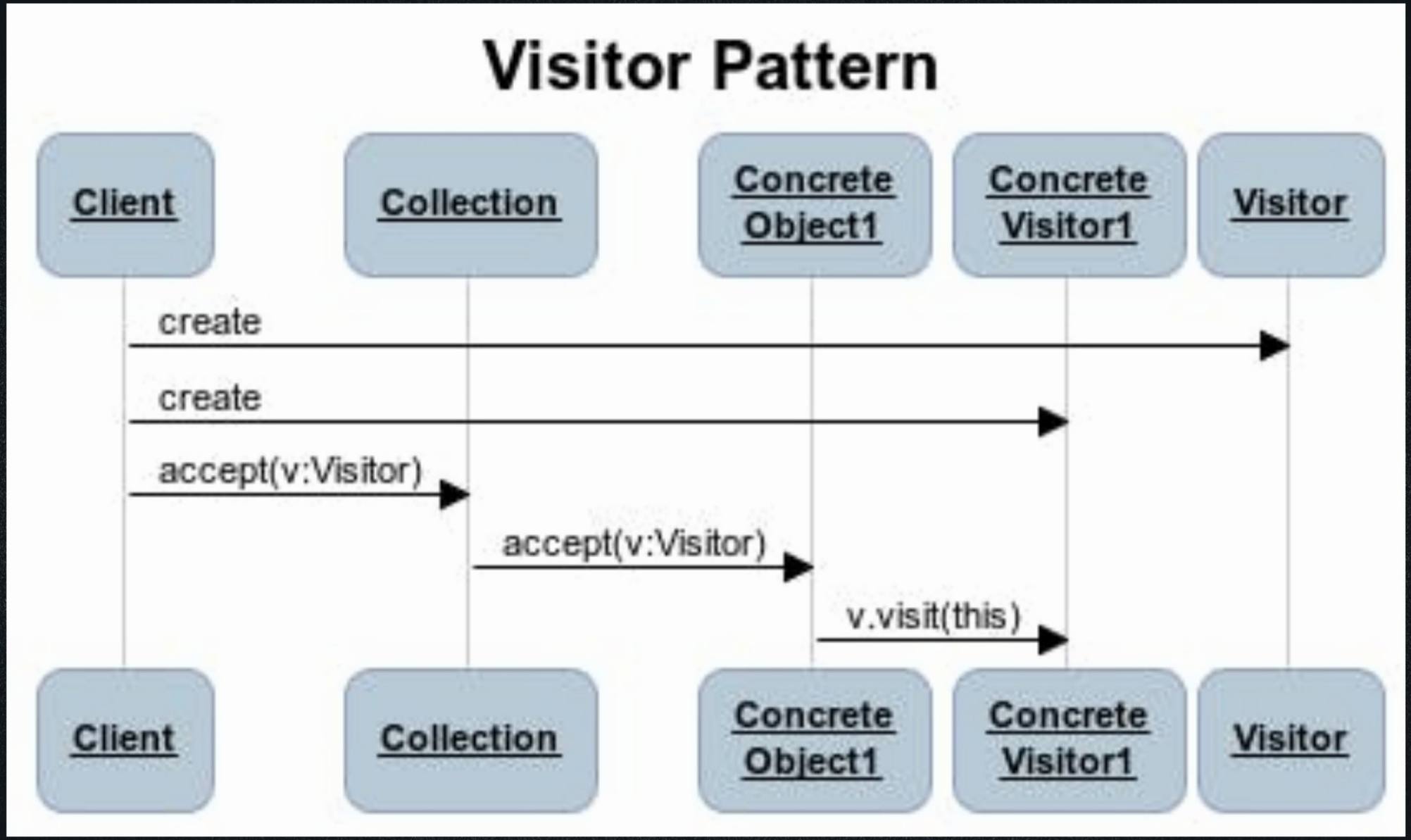
→ cand adaugam tipuri noi de obiecte foarte des

Mentimeter

Implementare

- cate o clasa de tip Visitor pentru fiecare operator
- o metoda accept in fiecare tip de obiect

Mentimeter



Visitor flow

Mentimeter

De ce nu apelam direct v.visit(element)?

- (in loc de accept(v))
- ne ajuta sa facem chestii mai fancy
- ex: arbori, vizitare recursiva pentru mai multe elemente

Mentimeter

Exemplu Visitor before & after

Mentimeter

```java
class Employee {
    String name;
    float salary;

    public Employee(String name, float salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public float getSalary() {
        return salary;
    }
}
class Manager extends Employee {
    float bonus;

    public Manager(String name, float salary) {
        super(name, salary);
        bonus = 0;
    }
    public float getBonus() {
        return bonus;
    }
    public void setBonus(float bonus) {
        this.bonus = bonus;
    }
}
```

Ierarhie obiecte

Mentimeter

```
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee( name: "Gigel",  salary: 100));
        employees.add(new Manager( name: "Ana",  salary: 1500));
    }
}
```

Lista de obiecte

Mentimeter

Daca vreau sa afisez bonusul managerilor?

- numai la runtime o sa stiu cine e manager si cine nu
- o solutie: instanceof (dar hai sa nu)

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee( name: "Gigel",  salary: 100));
        employees.add(new Manager( name: "Ana",  salary: 1500));

        for (Employee employee : employees) {
            if (employee instanceof Manager) {
                System.out.println("Bonus " + ((Manager) employee).getBonus() + "\n");
            } else {
                System.out.println("No bonus");
            }
        }
    }
}
```

instanceof

Mentimeter

Ce se schimba daca pun un tip nou de angajati?

- oriunde am verificat tipul trebuie sa mai bag inca un (if instanceof NouTip)
- cod greu de mentinut si usor de stricat

Mentimeter

Solutie

- vizitam "angajatii" si afisam venitul diferit in functie de tip

Mentimeter

```java
interface Visitable {
    public void accept(Visitor v);
}

class Employee implements Visitable {
    String name;
    float salary;

    public Employee(String name, float salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public float getSalary() {
        return salary;
    }
    public void accept(Visitor v) {
        v.visit( e: this);
    }
}
```

Visitable 1

Mentimeter

```java
class Manager extends Employee {
    float bonus;

    public Manager(String name, float salary) {
        super(name, salary);
        bonus = 0;
    }
    public float getBonus() {
        return bonus;
    }
    public void setBonus(float bonus) {
        this.bonus = bonus;
    }
    public void accept(Visitor v) {
        v.visit( m: this);
    }
}
```

Visitable 2

Mentimeter

```java
interface Visitor {
    public void visit(Employee e);
    public void visit(Manager m);
}

class BonusVisitor implements Visitor {
    @Override
    public void visit(Employee e) {
        System.out.println(e.getName() + " doesn't have a bonus.");
    }

    @Override
    public void visit(Manager m) {
        System.out.println(m.getName() + " has a bonus of " + m.getBonus());
    }
}
```

Visitor

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee( name: "Gigel",  salary: 100));
        employees.add(new Manager( name: "Ana",  salary: 1500));
        BonusVisitor visitor = new BonusVisitor();

        for (Employee employee : employees) {
            employee.accept(visitor);
        }
    }
}
```

Main

Mentimeter

Single dispatch

- referinta de tip Employee
- de-abia la runtime o sa stim exact ce tip avem (Employee vs Manager)

Mentimeter

Double dispatch

- depinde la runtime de 2 factori
- tipului elementului vizitat (Employee vs Manager)
- tipul vizitatorului (BonusVisitor)

Mentimeter

Aplicabilitate

- structuri complexe (obiecte si operatii diferite)
- simplificarea claselor (fiecare algoritm contine logica sa, nu trebuie sa ii bagam pe toti la un loc intr-o clasa)
- *adaugam rar obiecte noi de tip Visitable, dar adaugam des Visitor*

Mentimeter

Ce se intampla daca adaug un nou Visitable?

- fiecare Visitor deja existent trebuie modificat ca sa implementeze o noua metoda visit

Mentimeter

Ce se intampla daca adaug un nou Visitor?

- nu trebuie sa modific nimic din clasele deja existente
- doar imi definesc noua clasa

Mentimeter

Avantaje Visitor

- decuplam datele de operatii
- usureaza adaugarea de noi operatii
- poate gestiona obiecte de tipuri diferite
- poate mentine informatii de stare pe masura ce vizitam obiectele

Mentimeter

Dezavantaje Visitor

- depinde de stabilitatea ierarhiei
- tipuri noi de obiecte des + multe operatii => nu visitor
- daca avem nevoie de informatii de stare trebuie sa le expunem prin metode publice (altfel corupem incapsularea)